CSC 487 HW 3 Report – Sentiment Analysis

Code Explanation

In order to predict the sentiment—either positive or negative, so a binary classification problem—from texts loaded and annotated from an external dataset, I tried using two different approaches, the first of which being TF-IDF weighted histograms into a simple MLP model using sklearn. Only the top 1000 words were used, and the fitting of the TF-IDF vectorizer was done only on the training text to avoid data leakage from the test set into the model. A 90/10 train/test split was done on the corpus after it was preprocessed to remove break html tags and its labels remapped to 0 (negative) and 1 (positive). This resulted in 45000 train examples and 5000 test examples of texts. The default MLPClassifier was fit and used on these TF-IDF vectors, meaning one hidden layer of size 100, with ReLU activations, Adam as the optimizer, a learning rate of 1e-3, regularization with alpha=1e-4, and auto batching.

The second approach was using Byte Pair Encoding (BPE) tokenization and a Gated Recurrent Unit (GRU). The "bert-base-cased" pretrained auto tokenizer was used from Hugging Face to do the BPE tokenization of the corpus. Additionally, using a custom torch Dataset subclass, each text example was randomly truncated or zero padded if necessary to standardize each text length to a max sequence length, which was chosen as 100 tokens. The same 90/10 train/test split was used, with torch DataLoaders being used as to allow batching (standard batch size of 32) during training. A batch size of 1 was used in the test DataLoader to allow for different sequence lengths when calculating accuracy, as this would not be ok within the same batch. For the actual model, I had to implement a custom torch.nn.Module subclass, which included the vector embedding, GRU, and Linear layers in series. This was necessary to explicitly handle the tuple output that GRU provides, and extract out the final state of the GRU, which was then passed into the final Linear layer of the model to go from the hidden layer size down to two logits for the binary classification—positive or negative sentiment. An additional note is that the batch_first flag had to be set to True in the GRU for the batching to work. The hyperparameters provided in the assignment were then used to train the model, including Adam as the solver and using the CrossEntropyLoss function

I mostly used the pytorch documentation/APIs for guidance when completing this assignment, especially the examples for how the different classes and functions I needed were used, as well as which classes and functions were even available. For example, using the pytorch docs to figure out what the outputs from the GRU meant. I also used AI for double checking various bits of my code, such as using torchmetrics for calculating the

model accuracy or with my custom module for implementing the Embedding, GRU, and final linear classifier label in series.

Discussion

Converging after 89 epochs, the TF-IDF MLP setup resulted in a train accuracy of 100%, but only a test accuracy of 86.2%, meaning overfitting occurred. I then retrained this model with a higher alpha (so more regularization) by a factor of 10, though this still resulted in a train accuracy of 100% and a test accuracy of 86.5%, so there was barely any increase in performance. Thus, it seems that this simpler model would be unable to generalize well, even though it was able to get 100% train accuracy.

The BPE tokenization combined with the GRU performed much better, however resulting in final train and test accuracies of 89.9% and 90.9%, respectively after only 10 epochs of training. This was already more than 4% better test accuracy than the simple MLP model, showing that including a model that allows for inclusion of context from the rest of the text allows for greater generalization in sentiment classification. To see just how much better the test accuracy could get with the GRU, the number of training epochs was increased to 20, which resulted in train and test accuracies of 94% and 90%, respectively, suggesting that the GRU model could “memorize’ the whole train set just as the MLP did, though the best it can generalize to is about 90% accuracy. This cements the superiority of the GRU model over just a simple MLP for classifying sequence data like in this scenario.